

ESP32 WIFI CONTROLLED RC CAR

Advanced Robotics Project with Obstacle Detection

Technical Documentation & Design Analysis

Date: May 2026

Subject: Embedded Systems & Robotics

1. Project Overview

This documentation details the design and implementation of a two-wheel drive (2WD) robot car controlled via WiFi using the ESP32 microcontroller. The project integrates several fundamental engineering concepts, including wireless communication, pulse-width modulation (PWM) for motor control, and ultrasonic distance sensing for real-time autonomous safety.

1.1 Project Purpose

The primary goals of this project are to demonstrate mastery in:

- **Microcontroller Interfacing:** Connecting various peripherals (sensors, drivers, buzzers) to an ESP32.
- **Wireless Control:** Utilizing the ESP32's on-board WiFi to receive joystick commands.
- **Autonomous Safety:** Implementing a localized obstacle detection system that overrides user input to prevent collisions.
- **Power Management:** Effectively separating logic power from inductive load power to maintain system stability.

1.2 System Key Features

- **Differential Steering:** Precise navigation through varying motor speeds rather than mechanical steering racks.
- **Safety Interlock:** An ultrasonic sensor (HC-SR04) provides a 30cm safety buffer.
- **Signal Protection:** Use of a resistor-based voltage divider to protect 3.3V GPIO pins from 5V sensor signals.
- **User Feedback:** An active buzzer provides immediate auditory alerts when hazards are detected.

2. Hardware Specifications

The hardware was selected for its balance of performance, ease of integration, and cost-effectiveness for educational engineering purposes.

2.1 Component List

Component	Function	Specification
ESP32 DevKit	Central Processing Unit	38 Pin, Dual Core, 3.3V Logic
L298N Motor Driver	Actuator Controller	Dual H-Bridge, 2A Max per channel
HC-SR04	Ultrasonic Sensor	2cm - 400cm range, 5V supply
Active Buzzer	Alert System	High-level trigger
DC Gear Motors	Propulsion	3-6V Operating range
1kΩ & 2kΩ Resistors	Level Shifting	Voltage Divider for Echo pin

2.2 Power Architecture

To prevent electrical noise and brownouts, the system utilizes two separate power sources. This is a critical engineering standard in robotics.

- **Logic Supply:** A 5V power bank supplies the ESP32 via its micro-USB port. This provides clean, regulated power necessary for the WiFi radio and logic processing.

- **Motor Supply:** A dedicated battery pack connects directly to the L298N motor driver. This absorbs the high current spikes generated during motor stall or direction changes.
- **Common Ground:** All GND terminals are linked. This ensures that the 3.3V control signals from the ESP32 have a consistent reference point relative to the motor driver and sensors.

3. Electrical Interfacing

Correct wiring is vital for signal integrity. Below is the pinout configuration used in this build.

3.1 Pin Configuration Table

Component	Pin Description	ESP32 GPIO
Motor Driver	IN1 (Right Fwd)	GPIO27
Motor Driver	IN2 (Right Bkwd)	GPIO26
Motor Driver	IN3 (Left Fwd)	GPIO18
Motor Driver	IN4 (Left Bkwd)	GPIO19
Ultrasonic	Trig	GPIO23
Ultrasonic	Echo (Output)	GPIO32*
Buzzer	Positive (+)	GPIO25

**Note: GPIO32 is connected via a voltage divider node.*

3.2 Logic Level Protection

Important: The HC-SR04 operates at 5V logic. Connecting its Echo pin directly to the ESP32 (which is 3.3V tolerant) can damage the microcontroller. A voltage divider is implemented:

$$V_{\text{out}} = V_{\text{in}} \times [R2 / (R1 + R2)]$$
$$V_{\text{out}} = 5V \times [2k\Omega / (1k\Omega + 2k\Omega)] \approx \mathbf{3.33V}$$

4. Control Logic & Algorithms

4.1 Differential Steering Implementation

The car does not use a servo for steering. Instead, it utilizes differential steering logic. This allows for smoother turns while the vehicle is in motion.

- **Forward:** Both motors spin forward at equal speed.
- **Right Turn:** The left motor spins faster while the right motor spins slower, causing the car to arc to the right.
- **Left Turn:** The right motor spins faster while the left motor spins slower, causing the car to arc to the left.

4.2 Obstacle Detection Physics

The distance is calculated by measuring the time it takes for an ultrasonic pulse to travel to an object and bounce back.

$$\text{Distance} = (\text{Pulse Duration} \times 0.034) / 2$$

The constant 0.034 represents the speed of sound in cm/ μ s. The result is divided by 2 because the sound travels to the object and back.

4.3 Safety Algorithm

```
if (measuredDistance < 30) {  
    // Obstacle detected!  
    forwardBlocked = true;  
    stopMotors();  
    activateBuzzer(5000); // 5 second alert
```

```
} else {  
    forwardBlocked = false;  
}
```

Even when the forward path is blocked, the logic allows the user to move **backward** or **turn**. This ensures the robot does not become "stuck" and can be navigated away from the obstacle manually.

5. Conclusion

The ESP32 WiFi Controlled RC Car project successfully integrates hardware protection, wireless communication, and autonomous logic. By implementing a voltage divider, we ensure the longevity of the ESP32. The dual-power system ensures that the robotic movements do not interfere with the logic processing or WiFi stability.

This project serves as a robust foundation for more advanced robotics, such as adding camera feeds, line following capabilities, or integrating with IoT platforms for remote telemetry.